

React.js

SWEN-344 WEB ENGINEERING



Issues with DOM Manipulation

- DOM manipulation via JQuery or other vanilla JS does not scale well
 - No enforced class structure
 - Event handling is entirely up to you
 - Updating UI is entirely up to you
 - Leads to complex event handling routines
 - Single-Page Applications are difficult with this approach
 - No clean syntax for putting DOM definitions inside JS
- Response: SPA frameworks
 - e.g. React.js, Angular.js, Vue.js, Ember.js
 - Define everything as components
 - Define connections between them
 - Auto-update everything
 - Hardly ever refresh the (full) page

React.js

- Built by Facebook
- Used by over 380,000 sites currently
 - Source: <https://trends.builtwith.com/javascript/React>
- A sordid open source license history
 - Used to have concerning language around software patents
 - Today, React.js is MIT-Licensed (yay!)
 - Totally fine to use now
- Added syntax to Javascript, called **JSX**
 - Goal: specify DOM in HTML-ish syntax
 - Cleaner syntax, but can be a tad confusing
- Why React.js in this class?
 - Why not 😊
 - Plus, it's a common/ popular framework currently in use in the industry
 - Give a good intro into components

Key Design Decisions in a React.js App

- State should be **minimal**
 - You should never have to *sync* from one state to another
 - Instead, props **react** to changes in state
 - Similar concept of DRY and database normalization
- Where you hold your state
- Immutability leads to good design practices

JSX and component classes

JSX is an extended Javascript based language used by React.js

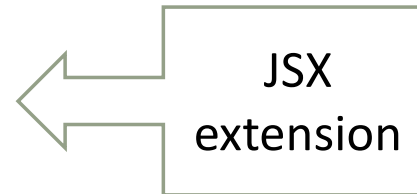
JSX enables writing html tags within javascript functions

When you see

```
class MyReactComponent extends React.Component
{
  render()
  {
    return (
      <div>SWEN-344 React Assignment</div>
    );
  }
}
```

```
export default MyReactComponent;
```

'class' allows component to retain state



Using functions: lambda (arrow) or standard

You can use class definitions for complex components, or use lambdas for simple HTML returns and stateless components

e.g.

```
import React from 'react';
const MyComponent =() =>
{
    #NOTE: No 'render' function
    return (
        <div>SWEN-344 React Assignment</div>
    );
}
export default MyComponent;
```

Or regular function definition

```
import React from 'react';
function MyComponent ()
{
    return (
        <div>SWEN-344 React Assignment</div>
    );
}
export default MyComponent;
```

Under The Hood

```
<div className="shopping-list">
  <h1>Shopping List for {this.props.name}</h1>
  <ul>
    <li>Instagram</li>
    <li>WhatsApp</li>
    <li>Oculus</li>
  </ul>
</div>
```

```
return React.createElement('div',
  {className: 'shopping-list'},
  React.createElement('h1', /* ... h1 children ...
  */),
  React.createElement('ul', /* ... ul children ...
  */),
);
```

React generates code from the JSX that you write

Anatomy of a React app

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import * as serviceWorker from './serviceWorker';
```

```
ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

If index.js is the file first loaded (is used to generate index.html)

- Index.js will render the component called `App`
- If you replace App with any other component, those will be rendered
- Uppercase tags are assumed to be React JSX

```
<React.StrictMode>
  <MyTextArea />
  <MyForm />
  <Pictures />
</React.StrictMode>,
```

Anatomy of a React app

```
<React.StrictMode>
  <MyTextArea />
  <MyForm />
  <Pictures />
</React.StrictMode>,
```

```
class MyTextArea extends React.Component {
  render() {
    return (
      <div>
        <div>SWEN-344 React Assignment</div>
        <textarea>This is my textarea</textarea>
      </div>
    );
  }
}
```

Anatomy of a React app

```
<React.StrictMode>  
  <MyTextArea />  
  <MyForm />  
  <Pictures />  
</React.StrictMode>,
```



The idea with React, is to create components for each control (or group of controls) and place them in separate .js files so they can render independently

Here, you would have 3 files:

- MyTextArea.js
- MyForm.js
- Pictures.js

Each with their own 'render' code

From html to React

One way to think of this is to take each major section of your html, and create a class with a render function, and move the html inside the render function

```
<body>

  <div>SWEN-344 React Assignment</div>
  <textarea>This is my textarea</textarea>
</div>
</body>
```



```
class MyHeading extends Component
{
  render() {
    return (
      <div>SWEN-344 React Assignment</div>
      <textarea>This is my textarea</textarea>
      </div>
    );
  }
}
```

```
//In App.js
function App() {
  return (
    <MyHeading />
  );}
```

Javascript?

End state: Convert to JSX events and arrow functions as event handlers

Step 1: (Just Starting out): You can add your old .js file in public/index.html, and call methods using:
onClick={window.functionName} or onClick =={() => window.functionName("param string")}

Step 2: (For Client-2): Convert to JSX events

```
doButtonFunction =() => {  
  //some code here.  This is a member fn. in the component class  
}  
  
...  
  
<button type="button" className="action"  
onClick={this.doButtonFn}>+</button>
```

State & Properties

- State is read-write
- Properties is read-only
- Every part of the UI is represented *once* in state.
 - For us, this generally means state is help by the top-level component
 - But, for bigger projects having mutually-exclusive states separately improves maintainability
- State gets passed to props on child components
 - Every component has a **render()**
 - Usually: render based on properties
- Child components update whenever the state changes
 - State changes trigger updates
 - Updates are all handled by a react runtime in the background

State

- State is very important in React. The only way react knows to re-render components (e.g., your webpage) is that it detects a change in state.
 - The primary way it detects changes in state is by looking at the state attributes you have defined and detect when their value is different than it was in a previous state
 - This can only be done if it knows (i.e., stores) what the prior state was
 - React encourages the use of `this.setState()`. This tells react that the state will change and, thus, a re-render is required
- State is private to components that it is defined in
 - Use props to pass information about state (future slide)

Setting State

```
class Board extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      squares: Array(9).fill(null),  
      xIsNext: true,  
    }  
  };  
}
```

Example of a class component with a state containing two attributes.

Board has two attributes in its **state**: *squares* and *xIsNext*. Whenever we change either of these, we should use `this.setState()`. For example:

```
this.setState({  
  squares: array(9).fill(5),  
  xIsNext: !this.state.xIsNext,  
})
```

This will inform react that the state is being modified so that it will re-render after we have finished modifying the state.

Properties

Properties is a way for a component to receive data (e.g., state data) from its parent.

```
renderSquare(i) {  
  return (  
    <Square  
      value={this.state.squares[i]}  
      onClick={() => this.handleClick(i)}  
    />  
  )  
}
```

Notice that square has two properties passed to it from its parent: value and onClick.

```
function Square(props) {  
  return (  
    <button  
      className="square"  
      onClick={props.onClick}  
    >  
      {props.value}  
    </button>  
  );  
}
```

Notice the 'props' parameter. We can access both onclick and value through 'props'.

Events

React supports different types of events

<https://reactjs.org/docs/events.html>

onClick

onContextMenu

onDoubleClick

Event naming convention is: on[Event]

NOTE: camelCase is required!

Also for 'className'

You can set these as properties just like in HTML

```
renderSquare(i) {  
  return (  
    <Square  
      value={this.state.squares[i]}  
      onClick={() => this.handleClick(i)}  
    />  
  )  
}
```